Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

# ЛАБОРАТОРНАЯ РАБОТА № 3

**«Разложение чисел на множители»**

по дисциплине «Теоретико-числовые методы в криптографии»

Выполнил Волошкевич М.А.
студент гр.5151001/40202 <подпись>

Преподаватель / Логачёва С.В.
ассистент <подпись>

Санкт-Петербург
2026г.

## 1. Цель работы.

Изучить и программно реализовать методы факторизации составных чисел: $\rho$-метод Полларда, $(p-1)$-метод Полларда, метод непрерывных дробей и метод квадратичного решета; исследовать эффективность методов на числах возрастающей размерности.

## 2. Теоретические сведения

Все реализованные методы решают задачу разложения составного числа $n$ на множители: найти нетривиальный делитель $p$ ($1 < p < n$), после чего делитель $q = n/p$ находится делением.

### 2.1. $\rho$-метод Полларда

Пусть $p$ — неизвестный делитель числа $n$ и $f$ — многочлен с целыми коэффициентами. Последовательность $x_{i+1} = f(x_i) \bmod n$ по модулю $p$ ведёт себя как случайная и в среднем за $O(\sqrt{p})$ шагов «зацикливается». По парадоксу дней рождения достаточно ждать коллизию $x_i \equiv x_j (\bmod p)$: тогда $\text{НОД}\big(|x_i - x_j|, n\big) > 1$.

Для поиска коллизии используется алгоритм Флойда: два указателя $a$ и $b$ движутся по последовательности, при этом $b$ делает два шага за итерацию, а $a$ — один; на каждой итерации проверяется $\text{НОД}(|a - b|, n)$. Трудоёмкость метода $O\big(n^{1/4}\big)$, память — $O(1)$.

### 2.2. $(p-1)$-метод Полларда

Пусть $n = pq$ и каноническое разложение $p - 1 = \prod p_i^{\alpha_i}$ содержит только малые простые. Выбирается база $B = \{p_1, ..., p_t\}$, включающая все простые $p_i \mid p - 1$, и вычисляется

$$M = \prod_{i=1}^{t} p_i^{[\ln n / \ln p_i]}.$$

Тогда $(p-1) \mid M$, и по малой теореме Ферма $a^M \equiv 1 (\bmod p)$ для любого $a$, взаимно простого с $p$. Значит, $d = \text{НОД}(a^M - 1, n) > 1$; при $a^{M\neg} \equiv 1 (\bmod n)$ получаем нетривиальный делитель. Метод эффективен, если $p - 1$ является $B$-гладким для небольшого $B$.

### 2.3. Метод Диксона: идея квадратичных сравнений

Общая идея последних двух методов: найти сравнение

$$s^2 \equiv t^2 (\bmod n),$$

то есть $(s + t)(s - t) \equiv 0 (\bmod n)$. Если $s^{\neg} \equiv \pm t (\bmod n)$, то $p = \text{НОД}(s - t, n)$ — нетривиальный делитель.

Для этого строится факторная база $B = \{p_0, p_1, ..., p_h\}$, где $p_0 = -1$, а $p_1, ..., p_h$ — простые. Находится $h + 2$ чисел $s_i$, для которых абсолютно наименьший вычет $s_i^2 \bmod n$ является $B$-гладким:

$$s_i^2 \equiv \prod_{j=0}^{h} p_j^{\alpha_{ij}} (\bmod n).$$

Каждому $s_i$ сопоставляется вектор показателей $(\alpha_{i0}, ..., \alpha_{ih})$. Произведение подмножества $K$ этих чисел является полным квадратом тогда и только тогда, когда все суммы показателей $\beta_j = \sum_{i \in K} \alpha_{ij}$ чётны, то есть векторы показателей по модулю 2 линейно зависимы над $\mathbb{F}_2$. Зависимость ищется методом Гауссова исключения над $\mathbb{F}_2$. Затем $s = \prod_{i \in K} s_i$, $t = \prod_j p_j^{\beta_j/2}$ и $s^2 \equiv t^2 (\bmod n)$.

### 2.4. Метод непрерывных дробей (CFRAC)

Числители $P_i$ подходящих дробей к $\sqrt{n}$ обладают свойством (следствие разложения $\sqrt{n}$ в непрерывную дробь):

$$P_i^2 - nQ_i^2 = (-1)^{i+1} Q_{i+1}, \quad \text{поэтому} \quad P_i^2 \bmod n = \pm Q_{i+1} < 2\sqrt{n}.$$

Так как $2\sqrt{n}$ много меньше $n$, вероятность $B$-гладкости вычета $P_i^2 \bmod n$ выше, чем у произвольного числа той же величины, поэтому CFRAC эффективнее метода Диксона со случайными $s_i$. Вычеты $P_i^2 \bmod n$ вычисляются рекуррентно:

$$P_{i+1} = a_i Q_i - P_i, \quad Q_{i+1} = \frac{n - P_{i+1}^2}{Q_i}, \quad a_i = \left[ \frac{P_i + \sqrt{n}}{Q_i} \right],$$

где $a_i$ — неполные частные, а $P_i$ — числители подходящих дробей.

### 2.5. Метод квадратичного решета (QS)

Выбирается многочлен $f(x) = \left(x + \left[\sqrt{n}\right]\right)^2 - n$. Для $|x| < M$ имеем $|f(x)| < 2\left[\sqrt{n}\right] M$, то есть значения многочлена малы. Вместо проверки каждого $f(x)$ на гладкость используется просеивание: для каждого простого $p$ из факторной базы решается сравнение

$$\left(x + \left[\sqrt{n}\right]\right)^2 \equiv n (\bmod p),$$

и его корни $x \equiv \pm x_p - \left[\sqrt{n}\right] (\bmod p)$ отмечаются в массиве (в log-варианте к ячейкам прибавляется $\ln p$). После просеивания кандидаты в $B$-гладкие — ячейки с большой накопленной суммой логарифмов — проверяются

пробными делениями. Просеивание резко сокращает число проверок по сравнению с CFRAC, где каждое $P_i^2 \bmod n$ проверяется индивидуально.

# 3. Ход работы

## 3.1. Программная реализация

Все методы реализованы на языке Python (приложение А) по учебнику Е. Б. Маховенко «Теоретико-числовые методы в криптографии» (глава 6): алгоритм 6.1 ($\rho$-метод), алгоритм 6.2 ($(p-1)$-метод), алгоритм 6.3 (метод Диксона) и разделы 6.4.1, 6.4.2 (CFRAC и QS). Корректность реализации проверена на контрольных примерах учебника: примеры 6.2, 6.4, 6.12, 6.13 дают те же делители (1181, 1201, 5531 и 2179 соответственно).

Общие для CFRAC и QS процедуры: построение факторной базы $B = \{-1\} \cup \{p \le B : (n/p) = 1\}$ (символ Лежандра вычисляется квадратичным законом взаимности), проверка $B$-гладкости пробными делениями, поиск линейной зависимости методом Гаусса над $\mathbb{F}_2$ (векторы показателей по модулю 2 хранятся битовыми масками) и вычисление $s$, $t$ по найденному множеству $K$. Если найденная зависимость даёт $s = \pm t \pmod n$ (тривиальное сравнение), проверяется следующая.

Исследуемые числа (задание №3, вариант из файла `2026_kvar6.txt`):

| № | Число $n$ | Знаков |
|---|---|---|
| 1 | 7725862768907051557 | 19 |
| 2 | 434154551935578441586027428350531478331 | 39 |
| 3 | 10854380007467828036094467779670889360795162191206049015516035227184511774880289 | 88 |

Таблица 1. Исследуемые числа

## 3.2. Число №1 (19 знаков)

### 3.2.1. $\rho$-метод Полларда

Параметры: $f(x) = x^2 + 1$, начальное значение $x_0 = c = 1$. Результат: делитель найден (Таблица 2).

| **Итерация $i$** | $a$ | $b$ | **НОД**$(\lvert a-b\rvert, n)$ |
|---|---|---|---|
| 1 | 2 | 5 | 1 |
| 2 | 5 | 677 | 1 |
| 3 | 26 | 210066388901 | 1 |
| 4 | 677 | 7290690510246855589 | 1 |
| 5 | 458330 | 7478792221548695530 | 1 |
| 56016 | 7183793499438470832 | 5909717857459534932 | 1 |
| 56017 | 2335596847830710897 | 804902650560252128 | 1 |
| 56018 | 2245668016100142065 | 6492937225281011753 | 1 |
| 56019 | 3938583173649550128 | 4448378945263257658 | 1 |
| 56020 | 5756122775572134580 | 3146215854544561852 | **6231597677** |

Таблица 2. $\rho$-метод, число №1: первые и последние значения

Делитель $p = 6231597677$ найден на итерации 56020; второй делитель $q = n/p = 1239788441$. Время работы: 1.06 сек (56020 итераций). Теоретическая оценка $O\big(n^{1/4}\big) \approx 5.2 \cdot 10^4$ итераций хорошо согласуется с фактическим числом итераций.

### 3.2.2. $(p-1)$-метод Полларда

Основание $a = 2$; показатели $l_i = [\ln n / \ln p_i]$. Результаты для разных границ базы (Таблица 3): при малых $B$ метод безрезультатен, так как разложение $p - 1 = 2^3 \cdot 5 \cdot 11 \cdot 2817701$ и $q - 1 = 2^2 \cdot 1557899419$ содержат большие простые множители.

| **Граница $B$** | $\lvert B\rvert$ | **Основание $a = 2^M \bmod n$** | **НОД**$(a-1, n)$ | **Время** |
|---|---|---|---|---|
| 100 | 25 | 5958249209388115067 | 1 | 0.01 сек |
| 1000 | 168 | 5306475177834914986 | 1 | 0.01 сек |
| 10000 | 1229 | 350135540020297244 | 1 | 0.01 сек |
| 3000000 | 216816 | 6964679719823752831 | **1239788441** | 1.51 сек |

Таблица 3. $(p-1)$-метод, число №1: результаты для разных баз

Делитель найден только при $B = 3 \cdot 10^6$, когда база покрывает простой множитель 2817701 числа $p - 1$.

### 3.2.3. Метод непрерывных дробей

Параметры: граница базы $B = 30000$. Факторная база содержит 1592 элемента (последний — 19993). Собрано 1594 гладких значения за 1.07 сек; поиск зависимости методом Гаусса — 0.19 сек.

Первые и последние гладкие значения (Таблица 4):

| $P_i$ | $P_i^2 \bmod n$ |
|---|---|
| 219583946455 | 3398299788 |
| 1106258363153 | 2233266381 |
| 2876827652923 | 646296604 |
| 6366424284052532827259 | 568795929 |
| 18361654020876148664274969 7 | 1034135076 |
| 10878396919637775336963435999943777594126420 33… | 1371872268 |
| 6762076423134803179321601838946772590547787 0… | 98628556 |
| 3456910862588260476340674322718536379678277 1… | 2214586371 |
| 5145811871576928579546462011643105699062440 4… | 1833387588 |
| 1601015083667549855143802035549276103918013 9… | 451532289 |

Таблица 4. CFRAC, число №1: первые и последние гладкие значения $P_i$

Первая и вторая найденные зависимости оказались тривиальными ($s = \pm t$); третья дала нетривиальный делитель:

$$s = 7168430529349481594, \quad t = 1519544788368767132,$$

$$s^2 \equiv t^2 \equiv 4247672934834276344 (\bmod n), \quad \text{НОД}(s - t, n) = 1239788441.$$

Полное время работы: 1.34 сек.

### 3.2.4. Метод квадратичного решета

Параметры: $B = 20000$, $M = 6 \cdot 10^7$, $f(x) = \left(x + \left[\sqrt{n}\right]\right)^2 - n$, $\left[\sqrt{n}\right] = 2779543626$, диапазон просеивания $|x| < 6 \cdot 10^7$. Факторная база — 1097 элементов. Просеивание заняло 29.4 сек, проверка кандидатов пробными делениями — 1247.6 сек, найдено 290170 гладких значений (нужно 1099). Поиск зависимости — 0.1 сек.

Первые и последние гладкие значения (Таблица 5):

| $x$ | $f(x)$ | Разложение $f(x)$ |
|---|---|---|
| −59999726 | −329943744879841557 | $-(3^4 \cdot 31 \cdot 743 \cdot 2293 \cdot 4133 \cdot 18661)$ |
| −59997898 | −329933802224001573 | $-(3^3 \cdot 13 \cdot 67^2 \cdot 1307 \cdot 9721 \cdot 16481)$ |
| −59997803 | −329933285510304228 | $-(2^2 \cdot 3 \cdot 43 \cdot 401 \cdot 6277 \cdot 12829 \cdot 19801)$ |
| −59997529 | −329931795199118148 | $-(2^2 \cdot 3^2 \cdot 53 \cdot 67 \cdot 113 \cdot 149 \cdot 9059 \cdot 16921)$ |
| −59997358 | −329930865114323733 | $-(3^3 \cdot 13 \cdot 271 \cdot 631 \cdot 991 \cdot 1531 \cdot 3623)$ |
| 59996303 | 337124239478273484 | $2^2 \cdot 3^2 \cdot 43 \cdot 163 \cdot 199 \cdot 593 \cdot 1063 \cdot 10651$ |
| 59996557 | 337125681964621932 | $2^2 \cdot 3 \cdot 31 \cdot 79 \cdot 281 \cdot 1877 \cdot 4409 \cdot 4933$ |
| 59996629 | 337126090858413468 | $2^2 \cdot 3 \cdot 163 \cdot 389 \cdot 6449 \cdot 8689 \cdot 7907$ |
| 59999670 | 337143360951492059 | $569 \cdot 1049 \cdot 2339 \cdot 12539 \cdot 19259$ |
| 59999843 | 337144343433502404 | $2^2 \cdot 3 \cdot 31 \cdot 401 \cdot 761 \cdot 809 \cdot 1601 \cdot 2293$ |

Таблица 5. QS, число №1: первые и последние гладкие значения $f(x)$

Первая зависимость ($K$ из 100 строк) дала нетривиальный делитель сразу:

$$s = 6045300309425099460, \quad t = 2799287296773996894,$$

$$s^2 \equiv t^2 \equiv 47541970620196067 10 (\bmod n), \quad \text{НОД}(s - t, n) = 1239788441.$$

Полное время работы: 1255.07 сек. Основные затраты — проверка кандидатов пробными делениями (1247.6 сек из 1255 сек).

### 3.3. Число №2 (39 знаков)

#### 3.3.1. $\rho$-метод Полларда

Параметры: $f(x) = x^2 + 1$, $c = 1$. Метод работал 2 часа (7200 сек) и выполнил 182981000 итераций, делитель не найден (Таблица 6): $\text{НОД}(|a - b|, n) = 1$ на всех итерациях. Ожидаемое число итераций $O\big(n^{1/4}\big) \approx 4.6 \cdot 10^9$; за 2 часа выполнено лишь $1.83 \cdot 10^8$ итераций ($\approx 4\%$), для успеха потребовалось бы около 50 часов. Метод признан нерезультативным на практике.

| Итерация $i$ | $a$ | $b$ | НОД |
|---|---|---|---|
| 1 | 2 | 5 | 1 |
| 2 | 5 | 677 | 1 |
| 3 | 26 | 210066388901 | 1 |
| 4 | 677 | 464192880877796116378286167933111116741 | 1 |
| 5 | 458330 | 328121120542657730573908170473471248834 | 42 |
| 182977000 | 363802697414861828586296109163041975338 | 280460591201287327389108000239382841206 | 106 |
| 182978000 | 285990093130312121471766830476990492165 | 509426493726104493120982785070087608331 | 31 |
| 182979000 | 323174645264403043751678477980537127830 | 386344523648906578938488525068849137997 | 97 |
| 182980000 | 256887104372481664220285934471465848734 | 430334535341579913053018062143394958772 | 72 |
| 182981000 | 975919619132632621164169736017772394582 | 299607745323928269427857881088071201001 | 101 |

Таблица 6. $\rho$-метод, число №2: первые и последние значения (7200 сек, без результата)

Ожидаемое число итераций $O\left(n^{1/4}\right) \approx 4.6 \cdot 10^9$; за 2 часа выполнено лишь $1.83 \cdot 10^8$ итераций ($\approx 4\%$), поэтому для успеха потребовалось бы около $4.6 \cdot \frac{10^9}{1.83 \cdot \frac{10^8}{7200}} \approx 50$ часов. Метод признан нерезультативным на практике.

### 3.3.2. $(p-1)$-метод Полларда

Основание $a = 2$. Разложения делителей: $p - 1 = 2 \cdot 3 \cdot 5 \cdot 13 \cdot 19 \cdot 41 \cdot 61 \cdot 9091 \cdot 5051 \cdot 21401$, $q - 1 = 2^4 \cdot 3 \cdot 5 \cdot 11 \cdot 13 \cdot 251 \cdot 21401 \cdot 5051 \cdot 25601$. Результаты (Таблица 7):

| Граница $B$ | $\lvert B \rvert$ | Основание $a = 2^M \bmod n$ | НОД$(a-1, n)$ | Время |
|---|---|---|---|---|
| 10000 | 1229 | 32727959582612805833555833292316282663 | 1 | 0.02 сек |
| 21401 | 2403 | 14305997738339217092333090482323800055 | **82118888930494588811** | 0.05 сек |
| 30000 | 3245 | 1 | $n$ (тривиальный) | 0.06 сек |

Таблица 7. $(p-1)$-метод, число №2: результаты для разных баз

При $B = 21401$ база покрывает все простые множители $p - 1$ (наибольший — 21401), и делитель $p = 82118888930494588811$ найден за 0.05 сек. При $B = 30000$ база покрывает уже и простые множители $q - 1$, поэтому $a^M \equiv 1 (\bmod n)$ и НОД тривиален: метод нашёл оба делителя одновременно.

### 3.3.3. Метод непрерывных дробей

Параметры: $B = 30000$; факторная база — 1638 элементов (последний — 29983). Собрано 1640 гладких значений за 115.06 сек; поиск зависимости — 0.23 сек. Первые и последние гладкие значения (Таблица 8):

| $P_i$ | $P_i^2 \bmod n$ |
|---|---|
| 135162057738208356858470149245297259115280 85 | 7137212485441821741 |
| 131636468314461545125882682140556842910190375719282… | 20425566322906438965 |
| 538449328995546785560964441414760284292685579465809… | 7447103353780951125 |
| 131473833919026417671773970530869737677124976518779… | 11774482619568821406 |
| 100766256212586539451974959696540999331507646371943… | 14983350859535466681 |
| 248214904278445584699771228229258472880644089246084… | 6862443007853402125 |
| 122842548665353612519778070992959525393267401876159… | 35285268897556872485 |
| 163734643484747400195884422791936517590164130074321… | 13187644223144805225 |
| 322994584474559047503972919905976975769406333736965… | 5240527617442429806 |
| 107837621444301028274288200790272286466920854944859… | 5150221113713201422 |

Таблица 8. CFRAC, число №2: первые и последние гладкие значения $P_i$

Две первые зависимости оказались тривиальными; третья дала делитель (Таблица 9):

| **Величина** | **Значение** |
|---|---|
| Множество $K$ | номера строк (1640 гладких чисел), подробно в логе `results/cfrac_n2.txt` |
| $s$ | 409818293782594498425932866967513966412 |
| $t$ | 262951335095699896916459544488484527702 |
| $s^2 \equiv t^2 (\mathrm{mod}\, n)$ | 166992366290567060350754996071270416069 |
| НОД$(s - t, n)$ | 23839073123744770321 |
| НОД$(s + t, n)$ | 18211888930494588811 |

Таблица 9. CFRAC, число №2: результат

Полное время работы: 116.39 сек.

### 3.3.4. Метод квадратичного решета

Параметры: $B = 20000$, $M = 6 \cdot 10^7$, $\lfloor \sqrt{n} \rfloor = 208363756909779883 52$. Факторная база — 1143 элемента. Просеивание и проверка — 72.88 сек, найдено 1452 гладких значения (нужно 1145); поиск зависимости — 0.12 сек.

Первые и последние гладкие значения (Таблица 10):

| $x$ | $f(x)$ |
|---|---|
| −59727832 | −248902313315470836794080793 1 |
| −59709337 | −248825239561790130114210810 6 |
| −59450986 | −247748619862565237779326037 5 |
| −59127525 | −246400668879093187113635440 2 |
| −59035747 | −246018204701460956029519230 6 |
| 59049049 | 246073629868238160054135647 0 |
| 59403502 | 247550733042801202821407898 5 |
| 59750614 | 248997244250574689102327082 5 |
| 59913907 | 249677731109718016870582475 0 |
| 59949353 | 249825444344266922892918869 4 |

Таблица 10. QS, число №2: первые и последние гладкие значения $f(x)$

Результат (Таблица 11):

| **Величина** | **Значение** |
|---|---|
| $s$ | 25028268786030428437453624129418978840 8 |
| $t$ | 51793171799097459983250706151182240116 |
| $s^2 \equiv t^2 \pmod n$ | 41591401360823520335550829242006867781 8 |
| НОД$(s-t, n)$ | 18211888930494588811 |
| НОД$(s+t, n)$ | 23839073123744770321 |

Таблица 11. QS, число №2: результат

Первая зависимость дала нетривиальный делитель. Полное время работы: 73.05 сек.

Предпринятые попытки: при $M = 2 \cdot 10^7$ гладких значений не хватило (701 из 1145), при уменьшении базы $B = 5000$, $M = 6 \cdot 10^7$ — тоже (72 из 322): слишком маленькая база резко снижает вероятность гладкости. Успешная комбинация — $B = 20000$, $M = 6 \cdot 10^7$.

### 3.4. Число №3 (80 знаков)

#### 3.4.1. $\rho$-метод Полларда

Параметры: $f(x) = x^2 + 1$, $c = 1$. Выполнено 5000000 итераций за 499.8 сек, делитель не найден. Ожидаемое число итераций $O\left(n^{1/4}\right) \approx 1.6 \cdot$

$10^{20}$ — метод нерезультативен принципиально: даже при 100 итераций/сек потребовалось бы $\approx 5 \cdot 10^{10}$ лет.

### 3.4.2. $(p-1)$-метод Полларда

Основание $a = 2$. Результаты (Таблица 12):

| **Граница $B$** | $\lvert B\rvert$ | **НОД$(a^M - 1, n)$** | **Время** |
|---|---|---|---|
| 10000 | 1229 | 1 | 0.09 сек |
| 1000000 | 78498 | 1 | 5.55 сек |
| 10000000 | 664579 | 1 | 46.55 сек |

Таблица 12. $(p-1)$-метод, число №3: результаты для разных баз

Метод нерезультативен: в разложениях $p-1$ и $q-1$ делителей числа №3 присутствуют большие простые (порядка $10^{11}$ и $10^{13}$), которые не входят ни в какую разумную базу. Увеличение $B$ в 1000 раз (от $10^4$ до $10^7$) увеличивает время лишь в 500 раз, но гладкость $p-1$ не достигается.

### 3.4.3. Методы CFRAC и QS

Факторная база для 80-значного числа при $B = 30000$ содержит около 3200 элементов; для сборки зависимости требуется не менее 3202 гладких значений. По экспериментальным данным для чисел №1 и №2 CFRAC находит $\approx 1500$ гладких в час, причём с ростом $n$ вероятность гладкости падает. Для числа №3 это означало бы время порядка $10^2 - 10^3$ часов только на сбор гладких значений.

Для QS ситуация ещё тяжелее: при $B \approx 50000$ база содержит около 5300 элементов, и из диапазона $|x| < 6 \cdot 10^7$ (просеивание которого занимает более 50 сек и требует $\approx 15$ ГБ памяти для массива кандидатов) набирается лишь единицы гладких значений. Для набора необходимого числа гладких нужно увеличивать $M$ и $B$ одновременно, что выводит время работы за пределы нескольких суток.

Оба метода признаны нерезультативными в разумное время. Для справки: факторизация числа №3 была получена методом эллиптических кривых (ECM, программа gmp-ecm) за минуты:

$$n_3 = 625742725193501219452965530711987283601 9 \cdot 1734639424551245017906665189752995482331,$$

оба множителя простые (проверено тестом Миллера–Рабина).

### 3.5. Сводные результаты

| **Число** | **$\rho$-метод** | **(p-1)-метод** | **CFRAC** | **QS** |
|---|---|---|---|---|
| №1 (19 зн.) | делитель за 1.06 сек, 56020 итер. | делитель при $B = 3 \cdot 10^6$, 1.51 сек | делитель за 1.34 сек | делитель за 1255.07 сек |
| №2 (39 зн.) | не найден за 7200 сек ($1.83 \cdot 10^8$ итер.) | делитель при $B = 21401$, 0.05 сек | делитель за 116.39 сек | делитель за 73.05 сек |
| №3 (80 зн.) | не найден (5000000 итер., 499.8 сек) | не найден ($B$ до $10^7$, 46.55 сек) | нерезультативен (время $\approx 10^2 - 10^3$ ч) | нерезультативен (время > неск. суток) |

Таблица 13. Сводные результаты работы методов

Полные факторизации:

$$n_1 = 1239788441 \cdot 6231597677,$$

$$n_2 = 18211888930494588811 \cdot 23839073123744770321,$$

$$n_3 = 625742725193501219452965530711987283601 9 \cdot 1734639424551245017906665189752995482331.$$

## 4. Выводы

В работе реализованы четыре метода факторизации: $\rho$-метод Полларда, $(p-1)$-метод Полларда, метод непрерывных дробей и метод квадратичного решета; корректность подтверждена на контрольных примерах учебника.

Эксперименты показали чёткое разделение методов по классам сложности. $\rho$-метод эффективен для чисел до 20 знаков (19-значное число факторизовано за секунду), но его сложность $O\left(n^{1/4}\right)$ делает бесполезным уже для 39-значных чисел. $(p-1)$-метод очень быстр (доли секунды), но работает лишь при $B$-гладкости $p-1$: для числа №2 потребовалась база до простого 21401, а для числа №3 метод неприменим ни при каком разумном $B$.

Методы класса квадратичных сравнений (CFRAC и QS) оказались наиболее универсальными: оба факторизовали и 19-значное, и 39-значное числа. CFRAC проще в реализации и не требует выбора параметра $M$, QS при удачном выборе параметров ($B = 20000$, $M = 6 \cdot 10^7$) оказался вдвое быстрее CFRAC на 39-значном числе (73 сек против 116 сек) благодаря просеиванию, но чувствителен к выбору $M$: при слишком малом $M$ гладких значений не хватает.

Для 80-значного числа ни один из реализованных методов не дал результата за приемлемое время: это согласуется с теорией — субэкспоненциальные методы CFRAC и QS имеют сложность $L_{n[1/2,c]}$, что для 80-значных чисел составляет сотни и тысячи часов вычислений; на практике такие числа факторизуются методом эллиптических кривых (ECM) или общим решетом числового поля (GNFS).

# Приложения

## Приложение А

main.py

```
import random
import math
import time
import sys

sys.set_int_max_str_digits(1000000)  # для печати больших чисел
P_i

#
=====================================================================
# расширенный бинарный алгоритм Евклида (из лабораторной работы
№1)
#
=====================================================================
def gcd(a, b):
    g = 1

    while a % 2 == 0 and b % 2 == 0:
        a = a // 2
        b = b // 2
        g = 2 * g

    u = a
    v = b
    A = 1
    B = 0
    C = 0
    D = 1

    while u != 0:
        while u % 2 == 0:
            u = u // 2
            if A % 2 == 0 and B % 2 == 0:
                A = A // 2
```

```
                B = B // 2
            else:
                A = (A + b) // 2
                B = (B - a) // 2

        while v % 2 == 0:
            v = v // 2

            if C % 2 == 0 and D % 2 == 0:
                C = C // 2
                D = D // 2
            else:
                C = (C + b) // 2
                D = (D - a) // 2

        if u >= v:
            u = u - v
            A = A - C
            B = B - D
        else:
            v = v - u
            C = C - A
            D = D - B

    d = g * v

    return d

#
# вспомогательные функции
#
def primes_up_to(B):
    # решето Эратосфена: все простые числа <= B
    sieve = [True] * (B + 1)
    sieve[0] = sieve[1] = False
    for i in range(2, int(B ** 0.5) + 1):
        if sieve[i]:
            for j in range(i * i, B + 1, i):
```

```
                sieve[j] = False
    return [i for i in range(2, B + 1) if sieve[i]]

def legendre_sym(a, p):
    # символ Лежандра (a/p) по квадратичному закону взаимности
    a = a % p
    t = 1
    while a != 0:
        while a % 2 == 0:
            a = a // 2
            if p % 8 in (3, 5):
                t = -t
        a, p = p, a
        if a % 4 == 3 and p % 4 == 3:
            t = -t
        a = a % p
    if p == 1:
        return t
    return 0

def factor_base(n, B):
    # факторная база B = {-1, p_1, ..., p_h}: простые p <= B, по
модулю
    # которых n является квадратичным вычетом, то есть (n/p) = 1
    base = [-1]
    for p in primes_up_to(B):
        if p == 2 or legendre_sym(n, p) == 1:
            base.append(p)
    return base

def is_B_smooth(x, base):
    # проверка B-гладкости числа x и построение вектора
показателей e;
    # если x не гладкое, возвращается None
    v = [0] * len(base)
    for i in range(len(base)):
        p = base[i]
        if p == -1:
            while x < 0:
                x = -x
```

```
                v[i] = v[i] + 1
        else:
            while x % p == 0:
                x = x // p
                v[i] = v[i] + 1
    if x == 1:
        return v
    return None

def sqrt_mod(a, p):
    # решение сравнения x^2 = a (mod p) (алгоритм Тонелли-Шенкса)
    a = a % p
    if p == 2:
        return 1
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    # общий случай Тонелли-Шенкса
    q = p - 1
    s = 0
    while q % 2 == 0:
        q = q // 2
        s = s + 1
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z = z + 1
    m = s
    c = pow(z, q, p)
    t = pow(a, q, p)
    R = pow(a, (q + 1) // 2, p)
    while t != 1:
        i = 0
        t2 = t
        while t2 != 1:
            t2 = t2 * t2 % p
            i = i + 1
        b = pow(c, 1 << (m - i - 1), p)
        m = i
        c = b * b % p
```

```
        t = t * c % p
        R = R * b % p
    return R

#
=============================================================
# поиск линейной зависимости методом Гаусса над F2 (шаг 3
алгоритма 6.3);
# векторы задаются битовыми масками, к каждому приписывается
единичный
# столбец с номером строки, по нулевой строке восстанавливается
множество K;
# возвращает до count независимых нулевых строк (зависимостей)
#
=============================================================
def find_dependency(vectors, k, count=3):
    m = len(vectors)
    rows = []
    for i in range(m):
        rows.append(vectors[i] | (1 << (k + i)))

    r = 0
    for c in range(k):
        # ищем строку с единицей в столбце c
        sel = -1
        for i in range(r, m):
            if (rows[i] >> c) & 1 == 1:
                sel = i
                break
        if sel == -1:
            continue
        rows[r], rows[sel] = rows[sel], rows[r]
        # гауссово исключение: обнуляем столбец c во всех
остальных строках
        for i in range(m):
            if i != r and (rows[i] >> c) & 1 == 1:
                rows[i] = rows[i] ^ rows[r]
        r = r + 1

    # нулевые строки дают искомые множества K
```

```
    deps = []
    for i in range(m):
        vec = rows[i] & ((1 << k) - 1)
        idx = rows[i] >> k
        if vec == 0 and idx != 0:
            # из маски делаем список номеров строк
            K = []
            for j in range(k + m):
                if (idx >> j) & 1 == 1:
                    K.append(j)
            deps.append(K)
            if len(deps) >= count:
                break
    return deps

#
# ро-метод Полларда (алгоритм 6.1)
#
def pollard_rho(n, c, f, max_iter=100000000):
    # алгоритм Флойда: a движется с удвоенной скоростью
    a = c
    b = c
    it = 0
    history = []

    while True:
        a = f(a) % n
        b = f(f(b) % n) % n
        d = gcd(abs(a - b), n)
        it = it + 1
        history.append((a, b, d))

        if it <= 5 or it % 1000 == 0:
            print("i =", it, " a =", a, " b =", b, " d =", d)

        if 1 < d < n:
            print("делитель найден:", d)
            return d, it, history
```

```
        if d == n:
            print("d = n, делитель не найден")
            return None, it, history

def run_rho(n, c, f):
    print("=== ро-метод Полларда: n =", n, " c =", c, " f(x) = x^2
+ 1 ===")
    t0 = time.time()
    d, it, history = pollard_rho(n, c, f)
    t = time.time() - t0
    print("число итераций:", it)
    print("время работы:", round(t, 2), "сек")
    print("первые 5 значений (i, a, b, НОД(a-b, n)):")
    for i in range(min(5, len(history))):
        print("  ", i + 1, history[i])
    print("последние 5 значений (i, a, b, НОД(a-b, n)):")
    for i in range(max(0, len(history) - 5), len(history)):
        print("  ", i + 1, history[i])
    return d, it, t

#
==========================================================================
# (p-1)-метод Полларда (алгоритм 6.2)
#
==========================================================================
def pollard_p_minus_1(n, base):
    # показатели l_i = [ln n / ln p_i]
    a = 2
    print("=== (p-1)-метод Полларда: n =", n, "===")
    print("размер базы:", len(base), "последний элемент:",
base[-1])

    ls = []
    for p in base:
        l = int(math.log(n) // math.log(p))
        ls.append(l)
        a = pow(a, p ** l, n)

    d = gcd(a - 1, n)
```

```
    print("основание a =", a)
    print("НОД(a - 1, n) =", d)
    if 1 < d < n:
        print("делитель найден:", d)
        return a, d, ls
    print("делитель не найден")
    return a, d, ls

def run_p_minus_1(n, B):
    base = primes_up_to(B)
    t0 = time.time()
    a, d, ls = pollard_p_minus_1(n, base)
    t = time.time() - t0
    print("время работы:", round(t, 2), "сек")
    print("значения l_i (уникальные):", sorted(set(ls)))
    return a, d, t

#
# ============================================================================
# шаги 4-5 алгоритма 6.3, общие для CFRAC и QS:
# по множеству K считаем s, t и делитель
#
# ============================================================================
def finish_dixon(n, K, s_list, exp_vectors, base):
    k = len(base)
    s = 1
    beta = [0] * k
    for i in K:
        s = s * s_list[i] % n
        for j in range(k):
            beta[j] = beta[j] + exp_vectors[i][j]

    print("множество K (номера строк):", K)
    print("суммарный вектор beta:", beta)

    # t = произведение p_j^(beta_j / 2)
    t = 1
    for j in range(k):
        if base[j] != -1 and beta[j] > 0:
            t = t * pow(base[j], beta[j] // 2, n) % n
```

```
    d1 = gcd(abs(s - t), n)
    d2 = gcd(s + t, n)
    print("s =", s)
    print("t =", t)
    print("s^2 mod n =", s * s % n)
    print("t^2 mod n =", t * t % n)
    print("НОД(s - t, n) =", d1)
    print("НОД(s + t, n) =", d2)

    if 1 < d1 < n:
        print("делитель:", d1)
        return d1
    if 1 < d2 < n:
        print("делитель:", d2)
        return d2
    print("s = ±t (mod n), тривиальная зависимость")
    return None

#
=====================================================================
# метод непрерывных дробей (алгоритм 6.3 + раздел 6.4.1)
#
=====================================================================
def CFRAC(n, B, max_iter=100000000):
    print("=== метод непрерывных дробей: n =", n, " B =", B,
"===")
    t0 = time.time()

    # шаг 1: база разложения
    base = factor_base(n, B)
    k = len(base)
    print("факторная база:", k, "элементов, последний:", base[-1])
    print("база:", base)

    # рекуррентные формулы для разложения sqrt(n) в непрерывную
дробь:
    # P_0 = 0, Q_0 = 1, a_i = [(P_i + sqrt(n)) / Q_i]
    # числители подходящих дробей: P_i = a_i * P_{i-1} + P_{i-2}
    P = 0
```

```
    Q = 1
    Pm1 = 1  # P_{-1}
    Pm2 = 0  # P_{-2}

    s_list = []        # числители P_i
    exp_vectors = []   # векторы показателей
    mods = []          # значения P_i^2 mod n

    for i in range(max_iter):
        # очередное неполное частное и числитель подходящей дроби
        a_i = (math.isqrt(n) + P) // Q
        Pi = a_i * Pm1 + Pm2
        Pm2 = Pm1
        Pm1 = Pi

        # рекуррентные формулы для P_{i+1}, Q_{i+1}
        P_next = a_i * Q - P
        Q_next = (n - P_next * P_next) // Q

        # P_i^2 mod n = |P_i^2 - n*Q_i^2| < 2*sqrt(n) (следствие теоремы 6.2),
        # при этом P_i^2 - n*Q_i^2 = (-1)^(i+1) * Q_{i+1}
        if i % 2 == 1:
            val = Q_next
        else:
            val = n - Q_next

        # проверка B-гладкости
        v = is_B_smooth(val, base)
        if v is not None:
            s_list.append(Pi)
            mods.append(val)
            exp_vectors.append(v)
            if len(s_list) <= 5 or len(s_list) % 100 == 0:
                print("гладкое №", len(s_list), ": i =", i, " P_i =", Pi,
                      " P_i^2 mod n =", val)

        P = P_next
        Q = Q_next
```

```
            # нужно h+2 гладких чисел
            if len(s_list) >= k + 2:
                break

    print("найдено гладких:", len(s_list), "нужно:", k + 2)
    print("время:", round(time.time() - t0, 2), "сек")

    print("первые 5 значений P_i и (P_i)^2 mod n:")
    for i in range(min(5, len(s_list))):
        print("   P_i =", s_list[i], "  (P_i)^2 mod n =", mods[i])
        print("   вектор:", exp_vectors[i])
    print("последние 5 значений P_i и (P_i)^2 mod n:")
    for i in range(max(0, len(s_list) - 5), len(s_list)):
        print("   P_i =", s_list[i], "  (P_i)^2 mod n =", mods[i])
        print("   вектор:", exp_vectors[i])

    if len(s_list) < k + 2:
        print("гладких чисел недостаточно")
        return None

    # шаг 3: векторы показателей по модулю 2 -> поиск зависимости
    masks = []
    for v in exp_vectors:
        mask = 0
        for j in range(k):
            if v[j] % 2 == 1:
                mask = mask | (1 << j)
        masks.append(mask)

    t1 = time.time()
    deps = find_dependency(masks[:k + 1], k)
    print("поиск зависимости занял", round(time.time() - t1, 2),
"сек")
    if len(deps) == 0:
        print("зависимость не найдена")
        return None

    # шаги 4-5: перебираем найденные зависимости, пока не получим
нетривиальный
```

```
    # делитель (первая может оказаться тривиальной: s = ±t (mod
n))
    for K in deps:
        d = finish_dixon(n, K, s_list, exp_vectors, base)
        if d is not None:
            print("полное время работы:", round(time.time() - t0,
2), "сек")
            return d
        print("зависимость оказалась тривиальной, пробуем
следующую")

    print("полное время работы:", round(time.time() - t0, 2),
"сек")
    return None

#
=============================================================
# метод квадратичного решета (алгоритм 6.3 + раздел 6.4.2)
#
=============================================================
def QS(n, B, M):
    # f(x) = (x + [sqrt(n)])^2 - n, просеиваем x из диапазона |x|
< M
    print("=== метод квадратичного решета: n =", n, " B =", B, " M
=", M, "===")
    t0 = time.time()

    # шаг 1: база разложения
    base = factor_base(n, B)
    k = len(base)
    print("факторная база:", k, "элементов, последний:", base[-1])
    print("база:", base)

    r = math.isqrt(n)
    print("[sqrt(n)] =", r, ", диапазон x: (-", M, ", ", M, ")",
sep="")

    # просеивание: для каждого p базы решаем сравнение (6.1):
    # (x + r)^2 = n (mod p), получаем два корня x = x_j - r (mod
p)
```

```
    # и отмечаем в массиве все такие x
    log_sieve = [0.0] * (2 * M + 1)
    for p in base:
        if p == -1:
            continue
        lp = math.log(p)
        x1 = sqrt_mod(n, p)
        if x1 is None:
            continue
        x2 = (p - x1) % p
        for xr in {x1, x2}:
            a0 = (xr - r) % p
            start = (a0 + M) % p
            while start < 2 * M + 1:
                log_sieve[start] = log_sieve[start] + lp
                start = start + p

    print("просеивание завершено за", round(time.time() - t0, 2),
"сек")

    # кандидаты: log f(x) ~ log(2 r M); так как решето учитывает
каждое
    # простое один раз, а показатель степени в разложении f(x)
может быть
    # больше 1, берём запас побольше и всех кандидатов проверяем
    # пробными делениями по факторной базе
    target = math.log(2 * r * M) - 2.5 * math.log(B)

    s_list = []        # значения s_i = x + r
    xs = []            # значения x
    exp_vectors = []   # векторы показателей
    mods = []          # значения f(x)

    for idx in range(2 * M + 1):
        if log_sieve[idx] > target - math.log(B) / 2:
            x = idx - M
            si = x + r
            fx = si * si - n
            v = is_B_smooth(fx, base)
            if v is not None:
```

```
                s_list.append(si)
                xs.append(x)
                mods.append(fx)
                exp_vectors.append(v)

    print("найдено гладких:", len(s_list), "нужно:", k + 2)
    print("время на просеивание и проверку:", round(time.time() -
t0, 2), "сек")

    print("первые 5 значений x и f(x):")
    for i in range(min(5, len(s_list))):
        print("   x =", xs[i], "  f(x) =", mods[i])
        print("   вектор:", exp_vectors[i])
    print("последние 5 значений x и f(x):")
    for i in range(max(0, len(s_list) - 5), len(s_list)):
        print("   x =", xs[i], "  f(x) =", mods[i])
        print("   вектор:", exp_vectors[i])

    if len(s_list) < k + 2:
        print("гладких чисел недостаточно")
        return None

    # шаг 3: векторы показателей по модулю 2 -> поиск зависимости
    masks = []
    for v in exp_vectors:
        mask = 0
        for j in range(k):
            if v[j] % 2 == 1:
                mask = mask | (1 << j)
        masks.append(mask)

    t1 = time.time()
    deps = find_dependency(masks[:k + 1], k)
    print("поиск зависимости занял", round(time.time() - t1, 2),
"сек")
    if len(deps) == 0:
        print("зависимость не найдена")
        return None

    # шаги 4-5; s_i = x_i + [sqrt(n)]; перебираем зависимости,
```

```
пока
    # не получим нетривиальный делитель
    for K in deps:
        d = finish_dixon(n, K, s_list, exp_vectors, base)
        if d is not None:
            print("полное время работы:", round(time.time() - t0,
2), "сек")
            return d
        print("зависимость оказалась тривиальной, пробуем
следующую")

    print("полное время работы:", round(time.time() - t0, 2),
"сек")
    return None

#
=====================================================================
# числа варианта
#
=====================================================================
N1 = 7725862768907051557
N2 = 434154551935578441586027428350531478331
N3 =
10854380007467828036094467779670889360795162191206049015516035227184511774880

if __name__ == "__main__":
    pass
```